

90 Days SDET Interview Prep

Day 11



Interview Questions

1. What is Select Class in Selenium?	2
2. What are the other ways to Handle Dropdowns (Without Select Class)	3
3. Write XPath to Find Login Button	4
4. Locators in Selenium	5
5. Difference findElement() vs findElements()	6
6. Different types of Waits in Selenium	7
7. Where Have You Used Explicit Wait?	9
8. What is Implicit Wait?	10

1. What is Select Class in Selenium?

The Select class in Selenium is a built-in class specifically designed to handle HTML dropdown elements (<select> tags). It provides convenient methods to select and deselect options from dropdowns.

Package Import

```
import org.openqa.selenium.support.ui.Select;
```

Creating Select Object

```
WebElement dropdown = driver.findElement(By.id("country"));  
Select select = new Select(dropdown);
```

Selection Methods

Method	Description
selectByVisibleText("India")	Selects option by the visible text displayed
selectByValue("IN")	Selects option by the value attribute
selectByIndex(2)	Selects option by index (starts from 0)
getOptions()	Returns all options as List<WebElement>
getFirstSelectedOption()	Returns currently selected option
isMultiple()	Returns true if multi-select dropdown

Example

```
WebElement countryDropdown = driver.findElement(By.id("country"));  
Select countrySelect = new Select(countryDropdown);  
countrySelect.selectByVisibleText("India");  
// OR  
countrySelect.selectByValue("IN");  
// OR  
countrySelect.selectByIndex(5);
```

Important Points

- ✓ Select class only works with <select> HTML tags
- ✓ Throws **UnexpectedTagNameException** if used on non-select elements
- ✓ For multi-select, use `deselectByVisibleText()`, `deselectAll()` methods

2. What are the other ways to Handle Dropdowns (Without Select Class)

Modern web applications often use custom dropdowns that are NOT built with `<select>` tags. **These use `<div>`, `<ul>`, `<li>`, or other elements styled to look like dropdowns.**

Method 1: Click and Select from List

```
// Step 1: Click on dropdown to open it
driver.findElement(By.id("dropdown-trigger")).click();
// Step 2: Click on desired option
driver.findElement(By.xpath("//li[text()='Option 1']")).click();
```

Method 2: Using Actions Class

```
Actions actions = new Actions(driver);
WebElement dropdown = driver.findElement(By.id("menu"));
actions.moveToElement(dropdown).click().perform();
WebElement option = driver.findElement(By.linkText("Option"));
actions.moveToElement(option).click().perform();
```

Method 3: Using JavaScript Executor

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("arguments[0].click();", dropdownElement);
js.executeScript("arguments[0].click();", optionElement);
```

Method 4: Using sendKeys for Autocomplete

```
WebElement searchDropdown = driver.findElement(By.id("search-box"));
searchDropdown.sendKeys("India");
// Wait for suggestions to appear
driver.findElement(By.xpath("//span[contains(text(),'India')]")).click();
```

When to Use Which Method

Dropdown Type	Recommended Approach
<code><select></code> tag	Use Select class
Custom <code><div></code> / <code></code> dropdown	Click and Select method
Hidden dropdown	JavaScript Executor
Autocomplete/Searchable	sendKeys + click on suggestion
Hover-based menu	Actions class with moveToElement

3. Write XPath to Find Login Button

Multiple XPath strategies can be used to locate a login button. The best approach depends on the HTML structure.

Sample HTML

```
<button id="login-btn" class="btn primary" name="loginButton">Login</button>
```

XPath Examples

XPath	Description
//button[@id='login-btn']	Using ID attribute
//button[text()='Login']	Using exact text
//button[contains(text(),'Login')]	Using partial text with contains()
//button[contains(@class,'primary')]	Using partial class match
//button[@name='loginButton']	Using name attribute
//button[contains(@id,'login')]	ID contains 'login'
//form[@id='loginForm']//button	Using parent-child relationship
//input[@type='submit' and @value='Login']	Multiple conditions with 'and'

XPath Functions Quick Reference

1. contains(@attr, 'value') - Partial attribute match
2. starts-with(@attr, 'value') - Attribute starts with value
3. text() - Match element text content
4. normalize-space() - Trim whitespace before matching

4. Locators in Selenium

Locators are used to identify and find web elements on a page. Selenium provides 8 types of locators.

Remember the Locator Class in WebDriver is By

All 8 Locator Types

Locator	Syntax	Best Used When
ID	By.id("email")	Element has unique ID (fastest)
Name	By.name("username")	Form elements with name attribute
Class Name	By.className("btn-primary")	Element has unique class
Tag Name	By.tagName("input")	Finding all elements of a type
Link Text	By.linkText("Click Here")	Exact anchor text match
Partial Link Text	By.partialLinkText("Click")	Partial anchor text match
CSS Selector	By.cssSelector("#email")	Complex selections, faster than XPath
XPath	By.xpath("//input[@id='email']")	Most flexible, handles complex DOM

Locator Priority (Best Practice)

1. ID - Most reliable and fastest
2. Name - Good for form elements
3. CSS Selector - Fast and versatile
4. XPath - Use when others won't work

CSS Selector vs XPath

Feature	CSS Selector	XPath
Speed	Faster	Slightly slower
Readability	More readable	Can get complex
Traverse up DOM	Cannot	Can (using parent::)
Text matching	Cannot match text	Can use text()
Browser support	All browsers	All browsers

5. Difference findElement() vs findElements()

Aspect	findElement()	findElements()
Return Type	Single WebElement	List<WebElement>
No Match Found	Throws NoSuchElementException	Returns empty list (no exception)
Multiple Matches	Returns first matching element	Returns all matching elements
Use Case	Unique elements (login button)	Multiple similar elements (list items)

Never Use Find Element for Modern Applications Like ReactJS better to use WebDriverWait

Example Usage

```
// findElement - returns single element
WebElement loginBtn = driver.findElement(By.id("login"));

// findElements - returns list
List<WebElement> links = driver.findElements(By.tagName("a"));
System.out.println("Total links: " + links.size());

// Using findElements to check if element exists (no exception)
List<WebElement> elements = driver.findElements(By.id("popup"));
if (elements.size() > 0) {
    elements.get(0).click(); // Element exists
}
```

Interview Tip

Use findElements() when you want to avoid NoSuchElementException.

Check list size to verify element presence without try-catch blocks.

6. Different types of Waits in Selenium

Waits handle synchronization between test script execution and web page loading. There are three types of waits in Selenium.

Implicit Wait

A global wait that applies to ALL findElement() calls. If element is not immediately found, WebDriver polls the DOM for the specified duration.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

- ✓ Set once, applies throughout the session
- ✓ Only waits for element presence in DOM
- ✓ Cannot wait for specific conditions (visibility, clickable)

Explicit Wait

Waits for a specific condition to be true before proceeding. More precise and flexible than implicit wait.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email")));
```

Common Expected Conditions

Condition	Description
visibilityOfElementLocated(By)	Element is visible on page
elementToBeClickable(By)	Element is visible and enabled
presenceOfElementLocated(By)	Element exists in DOM
invisibilityOfElementLocated(By)	Element not visible or not in DOM
textToBePresentInElement(elem, text)	Element contains specific text
alertIsPresent()	Alert dialog is present
frameToBeAvailableAndSwitchToIt()	Frame is available to switch
titleContains(String)	Page title contains text

The default polling interval in Explicit Wait (WebDriverWait) is 500 milliseconds.

This means WebDriverWait checks the condition every 0.5 seconds until either the condition is met or the timeout expires.

Fluent Wait

Most flexible wait. Allows custom polling interval and ability to ignore specific exceptions.

```
Wait<WebDriver> fluentWait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofMillis(500))
    .ignoring(NoSuchElementException.class);

WebElement element = fluentWait.until(driver -> {
    return driver.findElement(By.id("dynamic-element"));
});
```

Comparison Table

Feature	Implicit	Explicit	Fluent
Scope	Global	Specific element	Specific element
Condition	Presence only	Multiple conditions	Custom conditions
Polling	Default (500ms)	Default (500ms)	Customizable
Exception Handling	No	No	Yes (ignoring)
Recommended	Avoid mixing	Most common	Complex scenarios

7. Where Have You Used Explicit Wait?

Explicit wait is used in scenarios where elements load dynamically and timing is unpredictable.

Real Project Scenarios

1. Form Submissions: Waiting for success message or next page element after clicking submit.

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("success-msg")));
```

2. AJAX Calls: Waiting for data to load after dropdown selection.

```
select.selectByVisibleText("India");  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("states-list")));
```

3. Page Navigation: Waiting for specific element on new page after click.

```
loginBtn.click();  
wait.until(ExpectedConditions.titleContains("Dashboard"));
```

4. Modal/Popup Handling: Waiting for modal to appear before interaction.

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("modal")));
```

5. Element State Changes: Waiting for button to become clickable after form validation.

```
wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));
```

6. Loading Spinners: Waiting for loading indicator to disappear.

```
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("spinner")));
```

Your GitHub Signup Example

```
WebDriverWait wait = new WebDriverWait(wd, Duration.ofSeconds(10));  
// Each field loads dynamically, so explicit wait ensures element is visible  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email"))).sendKeys("email@test.com");  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("password"))).sendKeys("Pass123");
```

8. What is Implicit Wait?

Implicit wait tells WebDriver to poll the DOM for a specified duration when trying to find an element. It is set once and applies globally to all `findElement()` calls.

Syntax

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

How It Works

- ✓ WebDriver attempts to find the element immediately
- ✓ If not found, it waits and retries (polls the DOM)
- ✓ Continues until element is found OR timeout expires
- ✓ Throws `NoSuchElementException` if timeout expires

Why Avoid Implicit Wait?

- ✓ Only checks element presence, not visibility or clickability
- ✓ Applies globally - slows down tests when elements genuinely don't exist
- ✓ Mixing with explicit wait causes unpredictable timeout behavior
- ✓ Harder to debug timing issues
- ✓ No control over specific conditions per element

Best Practice Recommendation

- ✓ Use Explicit Wait instead of Implicit Wait.
- ✓ It provides precise control, better debugging, and handles specific conditions. Selenium documentation warns against mixing both wait types.